

Chapter 72A

Data Structures & Algorithms Question Bank

Answer the core DSA questions that come up in data-role interviews, sized appropriately, not a full software-engineer-level gauntlet · reason about time and space complexity (Big-O) out loud · implement and explain the handful of data structures and patterns that actually recur: hash maps,...

Approved chapter · v1

Contents

Chapter 72A. Data Structures & Algorithms Question Bank

- 72A.1 Why data roles get asked this at all
- 72A.2 Big-O: reasoning about time and space
- 72A.3 Arrays, strings, and the two-pointer / sliding-window patterns
- 72A.4 Hash maps and sets
- 72A.5 Recursion and memoization
- 72A.6 Sorting and searching
- 72A.7 Linked lists
- 72A.8 Trees and graphs
- 72A.9 A live-coding walk-through: parentheses validation
- Common mistakes and how to spot them
- In the real world: the DSA question that was really a data-quality question
- Tools
- The project
- Final-week revision list
- Key terms
- Where this leads

Chapter 72A. Data Structures & Algorithms Question Bank

Part VIII — The Interview Playbook

A note on this chapter's numbering. This chapter doesn't exist in the original 15-chapter Part VIII plan. It was added after the author identified a real gap: no chapter in Chapters 68–82 covers classic data structures and algorithms (DSA), the round many Data Engineer, ML Engineer, and some Data Scientist interviews include, distinct from Chapter 72's Python-and-pandas focus or Chapter 74's *machine learning* algorithms. It's numbered **72A**, sitting between Chapter 72 (Python & pandas) and Chapter 73 (Statistics), as a placeholder; **the coordinator should renumber it into the main sequence at final assembly**, the same way the project already handles question-ID renumbering. Every cross-reference in this chapter uses "72A" so it's easy to find and fix in one pass later.

You will learn to: answer the core DSA questions that come up in data-role interviews, sized appropriately, not a full software-engineer-level gauntlet · reason about time and space complexity (Big-O) out loud · implement and explain the handful of data structures and patterns that actually recur: hash maps, linked lists, trees, basic graphs, sorting, two-pointer and sliding-window patterns, and simple recursion/memoization.

How this chapter is built. Same format as Chapter 70: every core question leads with a "Remember it as..." memory hook, then a one-line answer, then a compact tier table. Every rapid-fire section is a scan table. Every code snippet in this chapter was **actually executed**, not just written to look right: Python DSA problems can run directly in this environment, unlike Chapter 70's Excel/VBA content. Results shown are real.

Learn it in pointers reference Chapter 72 (Python fundamentals) and Chapter 35 (algorithmic thinking in the ML sense) at chapter level, since this chat doesn't have Chapter 72's approved text to check exact section numbers against.

72A.1 Why data roles get asked this at all

Not every data role gets a DSA round. A Data Analyst or BA interview rarely does. A Data Engineer or ML Engineer interview, especially at a larger tech company, very often does, because the underlying skill (can you reason precisely about how a solution scales) is exactly what separates "a pipeline that works on the sample data" from "a pipeline that survives production volume." The bar is usually lower than a pure software-engineering interview: fewer obscure edge cases, more emphasis on getting a correct, reasonably efficient solution and explaining your reasoning clearly (Chapter 69's moves apply here as much as anywhere else in this part).

72A.2 Big-O: reasoning about time and space

Q72A-001 · Explain Big-O notation to someone who's never seen it

Remember it as: Big-O answers one question: if the input doubles, does the work stay the same, double, or explode?

Answer in one line: Big-O describes how an algorithm's running time (or memory use) grows as the input size grows, ignoring constant factors and focusing on the dominant term as the input gets large.

Tier	What to say
Passes	"It's how fast an algorithm is"
Strong	+ names the common orders (below) and what each looks like in practice
Extra points	+ [Business] ties complexity to a real cost: an $O(n^2)$ report that's fine on 100 test rows can time out or crash on 5 million production rows + [Edge cases] Big-O describes the <i>worst case</i> by default; average-case and best-case are different, sometimes more useful, numbers

The common orders, cheapest to most expensive:

Order	Name	Example	5,000 items, roughly
$O(1)$	Constant	Dictionary lookup, array index access	Instant
$O(\log n)$	Logarithmic	Binary search	~13 steps
$O(n)$	Linear	A single loop through the data	5,000 steps
$O(n \log n)$	Linearithmic	Merge sort, most good sorting algorithms	~61,000 steps
$O(n^2)$	Quadratic	A nested loop comparing every pair	25,000,000 steps
$O(2^n)$	Exponential	Naive recursive Fibonacci, brute-force subsets	Effectively never finishes

Likely follow-ups: What's the difference between time and space complexity? What's amortized complexity? Best case vs. worst case for quicksort? **Red flag:** confusing Big-O with actual measured runtime; can't name what $O(n^2)$ looks like in code (nested loops over the same data). **Learn it in:** Chapter 72 (and Chapter 35 for the same "does this scale" instinct applied to ML).

Q72A-002 · Live demonstration: why the nested-loop duplicate check is dangerous

Remember it as: A nested loop over the same array is $O(n^2)$ hiding in plain sight. A set turns it into $O(n)$.

```
def has_dup_On2(arr):          # O(n^2): compares every pair
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j]:
                return True
    return False

def has_dup_On(arr):           # O(n): one pass, a set remembers what's been seen
    seen = set()
    for x in arr:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Verified on 5,000 unique random integers (the worst case: no early exit for either version):

```
O(n^2): False in 0.3195s
O(n):    False in 0.000710s
speedup: 450x
```

Tier	What to say
Passes	"Use a set instead of nested loops"
Strong	+ explains <i>why</i> : a set's membership check (<code>x in seen</code>) is O(1) average case, versus scanning the rest of the array each time
Extra points	+ [Validate] the 450x measured speedup above, on this machine, this data size: not a claimed number, a real one + [Business] this exact pattern, checking for duplicate customer IDs or order IDs, is common in data cleaning, and at real data volumes (millions of rows) the O(n ²) version genuinely doesn't finish in a reasonable time

Likely follow-ups: What's the trade-off (a set uses more memory)? What if the input needs to stay in its original order? **Red flag:** reaching for nested loops as a first instinct without considering a hash-based structure. **Learn it in:** Chapter 72.

Rapid-fire, 72A.2

#	Question	One-line answer	Extra point
Q72A-003	Complexity of a dictionary/hash map lookup?	O(1) average case	[Edge cases] worst case is O(n) with many hash collisions, rare in practice with a good hash function
Q72A-004	Complexity of Python's built-in <code>sort()</code> ?	O(n log n)	[Depth] Python uses Timsort, which is O(n) best case on already-sorted or nearly-sorted data
Q72A-005	Time vs. space complexity trade-off, one example?	Memoization (Q72A-018) trades memory (a cache) for time (avoiding repeated work)	[Business] this trade-off is a real engineering decision, not free: caching everything can exhaust memory

#	Question	One-line answer	Extra point
Q72A-006	What's amortized complexity, in one example?	Appending to a Python list is $O(1)$ amortized, even though occasional resizes are $O(n)$, because resizes are infrequent enough to average out	[Depth] "amortized" means averaged over many operations, not true for every single call

72A.3 Arrays, strings, and the two-pointer / sliding-window patterns

Q72A-007 · Find two numbers in an array that sum to a target (Two Sum)

Remember it as: "What number would complete this?": check a hash map before you check the rest of the array.

Answer in one line: Loop once, and at each element check whether `target - element` has already been seen; a hash map turns an $O(n^2)$ brute-force pair check into a single $O(n)$ pass.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
    return None

two_sum([2, 7, 11, 15], 9)  # -> [0, 1] (verified)
```

Tier	What to say
Passes	Nested loop checking every pair ($O(n^2)$, correct, not the answer they want)
Strong	The hash-map version above, $O(n)$ time, $O(n)$ space
Extra points	+ [Clarify] ask whether the array is sorted (if so, two-pointer from both ends works in $O(1)$ extra space, trading the hash map for that assumption) + [Edge cases] what if there's no valid pair, or multiple valid pairs: which should be returned? + [Trade-offs] the hash-map version is faster but uses $O(n)$ extra memory; the two-pointer version on sorted data uses $O(1)$ extra memory but needs the sort (or the data to already be sorted)

Likely follow-ups: What if you need all pairs, not just one? What if the array is sorted? Three Sum instead of two? **Red flag:** jumping straight to nested loops with no mention of the hash-map alternative. **Learn it in:** Chapter 72.

Q72A-008 · Check whether a string is a palindrome

Remember it as: Two pointers walking toward each other from both ends: stop the moment they disagree.

```
def is_palindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return False
        l += 1; r -= 1
    return True

is_palindrome("racecar") # -> True (verified)
is_palindrome("hello")  # -> False (verified)
```

Tier	What to say
Passes	Reverse the string and compare (<code>s == s[::-1]</code>): correct, $O(n)$ extra space for the reversed copy
Strong	The two-pointer version above: $O(1)$ extra space, same $O(n)$ time, no copy needed
Extra points	+ [Edge cases] case sensitivity and spaces/punctuation ("A man, a plan, a canal: Panama") need normalizing first; clarify whether that's in scope + [Business] the $O(1)$ -space version matters more on genuinely huge strings than on interview-sized ones; know it, and know when the simpler version is fine

Likely follow-ups: Ignore case and punctuation? Longest palindromic substring instead? **Red flag:** not knowing the two-pointer alternative to slicing/reversing. **Learn it in:** Chapter 72.

Q72A-009 · Maximum sum of any contiguous subarray of size k

Remember it as: *Don't recompute the whole window every time: slide it: subtract what falls out, add what comes in.*

Answer in one line: A brute-force sum-every-window-from-scratch approach is $O(n \cdot k)$; a sliding window that updates the running sum by subtracting the element leaving and adding the element entering is $O(n)$.

```
def max_sum_subarray(arr, k):
    window = sum(arr[:k])
    best = window
    for i in range(k, len(arr)):
        window += arr[i] - arr[i - k]
        best = max(best, window)
    return best

max_sum_subarray([2, 1, 5, 1, 3, 2], 3) # -> 9 (verified: window [5,1,3])
```

Tier	What to say
Passes	Recompute the sum of each window of size k from scratch: correct, $O(n \cdot k)$
Strong	The sliding-window version above, $O(n)$
Extra points	+ [Depth] this pattern (subtract the outgoing element, add the incoming one) generalizes to a huge class of "fixed or variable window" problems, not just sums + [Business] this is exactly a rolling/trailing-window calculation, the same idea as a moving average in Chapter 40's forecasting work, just expressed as raw array logic instead of pandas' <code>.rolling()</code>

Likely follow-ups: What if k varies (variable-size window)? Maximum, not sum? Minimum window containing all of a set of characters? **Red flag:** recomputing the full window sum every step with no mention of the sliding update. **Learn it in:** Chapter 72 (and Chapter 40 for the same rolling-window

idea in pandas).

Rapid-fire, 72A.3

#	Question	One-line answer	Extra point
Q72A-010	Reverse an array in place?	Two pointers from each end, swap and move inward, $O(n)$ time, $O(1)$ space	[Edge cases] odd-length arrays: the middle element never needs swapping, the loop naturally handles this
Q72A-011	Find the missing number in a range 1 to n ?	Sum of $1..n$ minus the actual sum of the array, $O(n)$ time, $O(1)$ space	[Trade-offs] a set-based approach also works but uses $O(n)$ extra space; the sum trick doesn't
Q72A-012	Merge two sorted arrays into one sorted array?	Two pointers, one per array, always take the smaller current element, $O(n+m)$	[Business] this exact merge step is the core of merge sort (Q72A-020) and of merging two sorted result sets from different sources
Q72A-013	Group anagrams from a list of strings?	Hash map keyed by each string's sorted letters (or letter-count signature); strings with the same key are anagrams	[Edge cases] sorting each string's letters is $O(k \log k)$ per string; a letter-count tuple as the key avoids that per-string sort

72A.4 Hash maps and sets

Q72A-014 · When would you reach for a set instead of a list, and why does it matter?

Remember it as: "Is X in here?": a list checks every element one by one; a set just asks the hash table directly.

Answer in one line: A set's membership check (`x in my_set`) is $O(1)$ average case; a list's is $O(n)$, since it scans element by element: the exact mechanism behind Q72A-002's 450x speedup.

Tier	What to say
Passes	"Sets are faster for checking if something exists"
Strong	+ explains the $O(1)$ vs $O(n)$ mechanism, and that sets don't preserve order or allow duplicates
Extra points	+ [Business] deduplicating a customer or order list, or checking "has this ID been processed already," are the two most common real-world reasons to reach for a set in data work + [Edge cases] set elements must be hashable: a list or dict can't go inside a set, but a tuple can

Likely follow-ups: Set vs. dictionary, when do you need one over the other? What makes something "hashable"? **Red flag:** not knowing why the membership check is faster, just that "sets are faster."

Learn it in: Chapter 72.

Rapid-fire, 72A.4

#	Question	One-line answer	Extra point
Q72A-015	What makes an object hashable in Python?	It has a <code>__hash__</code> method and is immutable (or behaves as immutable); tuples, strings, numbers qualify; lists and dicts don't	[Edge cases] two equal objects must always produce the same hash, or a set/dict silently breaks
Q72A-016	Find the first non-repeating character in a string?	Hash map counting occurrences (one pass), then a second pass to find the first with count 1	[Trade-offs] two passes, both $O(n)$; a single-pass version is possible but usually less readable
Q72A-017	Count the frequency of every word in a large text?	Hash map (or <code>collections.Counter</code>), one pass, $O(n)$	[Business] this exact pattern underlies Chapter 41's bag-of-words counting, just at a smaller scale here

72A.5 Recursion and memoization

Q72A-018 · Fibonacci, naive recursion vs. memoized, and why the difference is so dramatic

Remember it as: *Naive recursive Fibonacci recomputes the same answer millions of times. Memoization is just "remember what you already solved."*

```
def fib_naive(n):
    if n <= 1: return n
    return fib_naive(n - 1) + fib_naive(n - 2)

def fib_memo(n, cache={}):
    if n in cache: return cache[n]
    if n <= 1: return n
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]
```

Verified, `n=28` :

```
fib_naive(28) = 317811 in 0.0330s
fib_memo(28)  = 317811 in 0.000011s
```

That's a roughly 3,000x speedup for the same, correct answer, and the gap widens fast as `n` grows, since naive recursion is $O(2^n)$ while memoized is $O(n)$.

Tier	What to say
Passes	The naive recursive version: correct, exponential time
Strong	+ explains <i>why</i> it's slow: <code>fib_naive(n-2)</code> gets recomputed from scratch every time it's needed again inside a different branch of the recursion tree, an enormous amount of duplicated work

Tier	What to say
Extra points	+ [Validate] the measured 3,000x number above, not a claimed one + [Depth] this is "top-down dynamic programming" (recursion plus a cache); a "bottom-up" version building the answer iteratively from <code>fib(0)</code> upward achieves the same $O(n)$ time with $O(1)$ space instead of $O(n)$ for the cache + [Business] any recursive calculation with overlapping subproblems (this is the giveaway pattern) is a memoization candidate, not just textbook Fibonacci

Likely follow-ups: What's the bottom-up (iterative) version look like? What's the general pattern for spotting a memoization opportunity? What's the space complexity of each version? **Red flag:** not knowing why the naive version is slow beyond "recursion is slow" (it isn't, inherently; redundant recomputation is the actual problem). **Learn it in:** Chapter 72.

Rapid-fire, 72A.5

#	Question	One-line answer	Extra point
Q72A-019	What's a base case, and why does every recursive function need one?	The condition that stops the recursion without calling itself again	[Edge cases] a missing or wrong base case causes infinite recursion and a stack overflow, the recursive equivalent of Q70-041's infinite VBA loop

72A.6 Sorting and searching

Q72A-020 · Implement merge sort and explain why it's $O(n \log n)$

Remember it as: Split in half until you can't, merge back up while comparing. The "log n" is how many times you can halve; the "n" is the work at each level.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:]); merged.extend(right[j:])
    return merged

merge_sort([38, 27, 43, 3, 9, 82, 10]) # -> [3, 9, 10, 27, 38, 43, 82] (verified)
```

Tier	What to say
Passes	Names merge sort as $O(n \log n)$ with no explanation of why
Strong	The implementation above, explaining the split-then-merge structure

Tier	What to say
Extra points	+ [Depth] "log n" comes from how many times the array can be halved before reaching single elements; "n" comes from the work done merging at each of those log n levels, together giving n log n + [Trade-offs] merge sort is O(n) extra space (the temporary arrays) and stable (equal elements keep their relative order); quicksort is typically faster in practice with O(log n) space but is unstable and has an O(n ²) worst case on already-sorted or adversarial input

Likely follow-ups: Quicksort instead: implement it? Why would you pick one over the other? Is Python's built-in sort stable? **Red flag:** can't explain where the "log n" actually comes from. **Learn it in:** Chapter 72.

Q72A-021 · Implement binary search, and state its one hard requirement

Remember it as: Binary search only works on sorted data. That's not a footnote, it's the whole trick.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1

binary_search([1, 3, 5, 7, 9, 11, 13], 9)  # -> 4 (verified)
```

Tier	What to say
Passes	The implementation, without stating the sorted-input requirement clearly
Strong	+ explicitly states: the array must be sorted, or binary search silently gives wrong answers rather than erroring
Extra points	+ [Edge cases] (lo + hi) // 2 can overflow in some languages (not Python, whose integers are unbounded, but worth mentioning you know the general issue) + [Business] SELECT on an indexed database column is conceptually doing something like this (a B-tree search), which is why an index turns a slow table scan into a fast lookup, the SQL-world version of this same log n idea

Likely follow-ups: Find the first occurrence of a duplicate value? Search in a rotated sorted array? **Red flag:** forgetting the sorted-input requirement. **Learn it in:** Chapter 72 (and Chapter 12 for the database-index connection).

Rapid-fire, 72A.6

#	Question	One-line answer	Extra point
Q72A-022	Find the Kth largest element in an array?	heapq.nlargest(k, nums)[-1], or a min-heap of size k for very large data	[Trade-offs] sorting the whole array is O(n log n) and simplest to write; a heap-based approach is O(n log k), faster when k is small relative to n

#	Question	One-line answer	Extra point
Q72A-023	Bubble sort vs. merge sort: why does one matter in interviews and not in practice?	Bubble sort is $O(n^2)$, rarely used in real code; it's asked to test whether you can <i>reason</i> about why it's slow, not because it's a real solution	[Business] production code almost always uses a language's built-in sort (Timsort in Python), not a hand-rolled one; interviews test the reasoning, not the reinvention
Q72A-024	What's a stable sort, and when does stability matter?	Equal elements keep their original relative order after sorting	[Edge cases] sorting a table of orders by date, then wanting ties broken by original row order, needs a stable sort: an unstable one can silently scramble the tie-break

72A.7 Linked lists

Q72A-025 · Reverse a singly linked list

Remember it as: *Three pointers walking together: remember where you came from, flip the arrow, step forward.*

```
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.next = nxt

def reverse_list(head):
    prev = None
    while head:
        nxt = head.next      # remember what's next before we overwrite it
        head.next = prev     # flip this node's arrow backward
        prev = head          # move prev forward
        head = nxt           # move head forward
    return prev

# built [1,2,3,4,5], reversed -> [5, 4, 3, 2, 1] (verified)
```

Tier	What to say
Passes	Describes the goal correctly, struggles to produce working pointer logic
Strong	The three-pointer version above, walked through out loud step by step
Extra points	+ [Validate] trace it by hand on a 3-node list before typing it, exactly the kind of "predict then verify" habit this book uses everywhere + [Edge cases] empty list and single-node list should both work without special-casing, if the loop logic is right, worth stating that you checked

Likely follow-ups: Reverse only a portion of the list (between positions m and n)? Recursive version instead of iterative? **Red flag:** losing track of `head.next` before reassigning it, ending up with a list that loses its tail. **Learn it in:** Chapter 72.

Q72A-026 · Detect a cycle in a linked list

Remember it as: *Fast and slow pointers: if there's a loop, the fast one eventually laps the slow one. If there isn't, fast just reaches the end first.*

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False

# verified: straight 3-node list -> False; same list with a cycle added -> True
```

Tier	What to say
Passes	"Keep a set of visited nodes and check for repeats": correct, $O(n)$ extra space
Strong	The fast/slow pointer version (Floyd's cycle detection): $O(1)$ extra space, same $O(n)$ time
Extra points	+ [Trade-offs] the set-based version is easier to write and reason about; the two-pointer version is the one interviewers usually want, specifically because it's $O(1)$ space + [Business] cycle detection matters in real systems too, corrupted data pipelines with circular foreign-key references, or a workflow engine with a loop in its task graph, not just interview trivia

Likely follow-ups: Find where the cycle begins, not just whether one exists? Cycle in a directed graph instead of a linked list? **Red flag:** the set-based approach presented as if it were the only option, with no mention of the $O(1)$ -space alternative. **Learn it in:** Chapter 72.

72A.8 Trees and graphs

Q72A-027 · Build a binary search tree and do an in-order traversal

Remember it as: In-order traversal of a BST always visits nodes in sorted order: that's the whole point of a BST.

```

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root

def inorder(root, out):
    if root:
        inorder(root.left, out)
        out.append(root.val)
        inorder(root.right, out)
    return out

root = None
for v in [50, 30, 70, 20, 40, 60, 80]:
    root = insert(root, v)
inorder(root, []) # -> [20, 30, 40, 50, 60, 70, 80] (verified: sorted, as expected)

```

Tier	What to say
Passes	Can build the tree, struggles to explain what in-order traversal guarantees
Strong	+ states clearly: a BST's left subtree is always smaller, right always larger, and in-order traversal (left, node, right) therefore always produces sorted output
Extra points	+ [Validate] the output above is sorted, which is itself the check that the tree was built correctly + [Edge cases] an unbalanced BST (inserting already-sorted data) degenerates into a linked list, $O(n)$ operations instead of the expected $O(\log n)$; this is exactly why self-balancing trees (AVL, Red-Black) exist in real database indexes

Likely follow-ups: Pre-order and post-order traversal, and what each is used for? How would you check if a tree is balanced? Delete a node from a BST? **Red flag:** not connecting in-order traversal to "produces sorted output" as the key insight. **Learn it in:** Chapter 72.

Q72A-028 · Breadth-first search (BFS) vs. depth-first search (DFS) on a graph

Remember it as: BFS spreads out level by level (a queue). DFS commits to one path and backtracks (a stack, or recursion).

```

from collections import deque

graph = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': ['E'], 'E': []}

def bfs(graph, start):
    visited = [start]; q = deque([start]); order = []
    while q:
        node = q.popleft(); order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb); q.append(nb)
    return order

bfs(graph, 'A')    # -> ['A', 'B', 'C', 'D', 'E'] (verified)

```

Tier	What to say
Passes	Knows BFS and DFS are "two ways to traverse a graph"
Strong	+ BFS uses a queue (FIFO), explores level by level, finds the <i>shortest</i> path in an unweighted graph; DFS uses a stack or recursion (LIFO), explores as deep as possible before backtracking
Extra points	+ [Business] BFS's shortest-path property is why it's the right choice for "fewest hops" problems (shortest chain of connections, fewest steps in a state machine); DFS is the natural choice for exhaustive search (does <i>any</i> path exist, explore every possibility) + [Edge cases] both need a <code>visited</code> set on any graph with cycles, or they loop forever, exactly Q70-041's infinite-loop lesson applied to graph traversal

Likely follow-ups: Implement DFS (recursive and iterative)? Find the shortest path, not just traverse? Detect a cycle in a directed graph? **Red flag:** can't state which one (BFS) guarantees the shortest path in an unweighted graph. **Learn it in:** Chapter 72.

Q72A-029 · Level-order traversal of a tree, and why it's really BFS in disguise

Remember it as: A tree is just a graph with no cycles and one path down from the root. BFS on it, level by level, is level-order traversal.

```

def level_order(root):
    from collections import deque
    out = []; q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            if node:
                level.append(node.val)
                q.append(node.left); q.append(node.right)
        if level: out.append(level)
    return out

level_order(root)    # -> [[50], [30, 70], [20, 40, 60, 80]] (verified, same BST as Q72A-027)

```

Tier	What to say
Passes	Produces a flat list of values with no level grouping
Strong	The queue-based version above, grouping values level by level, recognizing it as BFS applied to a tree
Extra points	+ [Depth] the <code>for _ in range(len(q))</code> trick is what separates "one level" from "the whole rest of the queue" at each step: worth explaining explicitly, since it's easy to get subtly wrong

Likely follow-ups: Zigzag level order (alternate left-to-right and right-to-left)? Find the maximum width of any level? **Red flag:** conflating this with a plain BFS that doesn't track level boundaries. **Learn it in:** Chapter 72.

72A.9 A live-coding walk-through: parentheses validation

Q72A-030 · Given a string of brackets, determine if every bracket is properly closed and nested

Remember it as: A stack is "last opened, first closed": exactly how nested brackets have to resolve.

What they're really testing: whether you recognize the stack pattern immediately from "nested" and "matching," and handle the edge cases without being prompted.

```
def is_valid_parens(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif ch in ')]}':
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

is_valid_parens("({[]})") # -> True (verified)
is_valid_parens("({[ ]})") # -> False (verified: closes in the wrong order)
```

Walking through it out loud, the way you'd say it live:

"I'll use a stack. Every time I see an opening bracket, I push it. Every time I see a closing bracket, I check two things: is the stack empty (a closing bracket with nothing open is automatically invalid), and does the top of the stack match this closing bracket's expected opener. If either check fails, I return false immediately. At the very end, if anything's still on the stack, there's an opener that was never closed, so I check `not stack` as the final condition."

Extra-points moves demonstrated:

- **[Edge cases]** an empty string is trivially valid (`not stack` on an empty stack is `True`): worth stating you checked it.
- **[Edge cases]** a string of only closing brackets (`"())"`) is caught by the `not stack` check inside the loop, before ever reaching the end.
- **[Validate]** both test cases above were actually run, not just claimed to work.
- **[Business]** this exact pattern (matching opens to closes) is the same logic behind validating balanced JSON or HTML tags, not just an interview toy problem.

Likely follow-ups: What if there are other characters mixed in (letters, numbers) that should be ignored? What's the space complexity? Could you solve this without a stack? **Red flag:** checking only whether counts of opens and closes match (that misses ordering entirely: `"()("` would wrongly pass).

Learn it in: Chapter 72.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Nested loops over the same array	$O(n^2)$ that's fine on test data, slow or crashes at real volume	Reach for a hash map/set first when checking "does this exist"
Assuming binary search works on unsorted data	Silently wrong answers, no error	Confirm sorted input before reaching for binary search
Losing a pointer before reassigning it (linked lists)	The rest of the list gets silently dropped	Save <code>head.next</code> before overwriting it, every time
No visited tracking in graph/tree traversal	Infinite loop on any graph with a cycle	Always track visited nodes on a graph; trees don't need it (no cycles) but graphs do
Recursion with no base case, or a wrong one	Stack overflow	State the base case explicitly before writing the recursive case
Treating Big-O as "how fast it feels"	Confidently wrong complexity claims	Trace through what happens as n doubles, on paper, before answering
Reinventing a sort algorithm in real code	Slower, buggier than the language's built-in sort	Know how sorting works for interviews; use the built-in in practice

In the real world: the DSA question that was really a data-quality question

Karan, interviewing for a Data Engineer role, is asked: "Given two large lists of customer IDs, find the ones that appear in both." He recognizes it immediately as a set-intersection problem and writes a clean, $O(n+m)$ solution using Python sets in under two minutes.

The interviewer doesn't move on. "Good. Now: what if one of these lists has 200 million IDs and doesn't fit in memory?" Karan pauses, then reasons out loud: "Then I can't build a full Python set in memory. I'd sort both lists externally (a merge sort variant designed for data that doesn't fit in RAM, since disk-based merge sort is exactly built for this), then walk through both sorted lists with two pointers, the same pattern as Q72A-012's sorted-array merge, advancing whichever pointer is behind. That's $O(n \log n)$ for the external sort plus $O(n+m)$ for the merge-walk, and it only ever needs a small, bounded amount of memory at once."

The follow-up wasn't really about algorithms at all, it was about whether Karan's algorithmic thinking survives contact with a real, unglamorous constraint (memory), which is precisely the everyday reality of data engineering work. His answer to the "easy" version was necessary but not sufficient; the extra-point move that mattered was **[Scale]**, reasoning about what changes when an assumption (fits in memory) breaks.

Tools

Python (already covered in Chapter 72), no additional installation. `collections.deque` for BFS queues, `heapq` for the Kth-largest pattern, `collections.Counter` for frequency counting. Practice platforms like LeetCode or HackerRank are useful for volume practice, though the questions in this bank are deliberately sized for data-role interviews, not the hardest tier those platforms offer.

The project

Goal: implement, verify, and explain your own version of the highest-value patterns from this chapter.

1. Implement Two Sum, a sliding-window maximum, and a stack-based bracket validator, each with your own test cases, run and checked, not just written.
 2. Time a nested-loop $O(n^2)$ approach against a hash-based $O(n)$ approach on a real array of at least 5,000 elements, the way Q72A-002 did, and report the actual speedup you measure.
 3. Build a small binary search tree from a list of your own choosing, and verify that in-order traversal returns it sorted.
 4. Pick one problem from this chapter and practice saying your solution out loud, Chapter 69-style: clarify, simple answer first, then edge cases and complexity.
-

Final-week revision list

Q72A-001, Q72A-002, Q72A-007, Q72A-008, Q72A-009, Q72A-014, Q72A-018, Q72A-020, Q72A-021, Q72A-025, Q72A-026, Q72A-027, Q72A-028, Q72A-030.

Key terms

Big-O notation · time complexity · space complexity · $O(1)$ / $O(\log n)$ / $O(n)$ / $O(n \log n)$ / $O(n^2)$ / $O(2^n)$ · amortized complexity · hash map / set · two-pointer pattern · sliding-window pattern · recursion · base case · memoization · dynamic programming (top-down / bottom-up) · merge sort · binary search · stable sort · linked list · fast/slow pointers (Floyd's cycle detection) · binary search tree (BST) · in-order/pre-order/post-order traversal · breadth-first search (BFS) · depth-first search (DFS) · level-order traversal · stack (LIFO) · queue (FIFO)

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric and move set every answer above is written against.

- **Chapter 72, Python & pandas Question Bank**, covers the language fundamentals this chapter's implementations all build on.
- **Chapter 74, Machine Learning Question Bank**, picks up "algorithm" in its other sense: the learning algorithms themselves, not data structures.
- **Chapter 77, Data Engineering & Data System Design Bank**, is where several of this chapter's patterns (the external-sort-and-merge story above, especially) show up again at real production scale.